

RAG Copilot Week 3 专属执行手册

只服务 *enterprise-rag-copilot* 的维护、修复、补强与 Week 4 包装准备

Chenning + ChatGPT

生成日期：2026-06-10

文档属性	说明
适用项目	enterprise-rag-copilot：可迁移企业级 RAG Copilot。
Week 3 定位	RAG 不是主开发线；本周只做维护、失败修复、文档补强、指标清理和 Week 4 包装准备。
执行对象	Chenning 人工 owner + Codex 候选实现生成者。Codex 不拥有最终合并权。
最高约束	完全遵守 Vibe Coding 项目实践公约：AI 输出是候选稿，主线任务先定接口，维护任务必须基于 failure case，性能指标必须可复现。
核心交付	至少修复 3 个高优先级 RAG failure cases；README / architecture / evaluation / performance / failure_cases 补强；准备 2 分钟 RAG 演示；至少 2 条 RAG 维护 commit。

本 PDF 的写法刻意面向两类读者：Chenning 能知道每天要做什么；Codex 能知道每天允许改哪些文件、不能改什么、怎么验收、什么情况下拒绝合并。

0. 静态目录

- 1. 文档定位与 Week 3 RAG 核心判断
- 2. 实践公约在 Week 3 RAG 维护中的落地方式
- 3. Week 3 RAG 总交付物与目录边界
- 4. Codex 工作协议：每次任务必须遵守的上下文、约束、验收
- 5. Day 1 - Failure Triage：整理 Week 2 失败样例
- 6. Day 2 - Intent / Fallback Fix：修复高优先级路由与兜底问题
- 7. Day 3 - Retrieval / Metadata Fix：修复检索与 metadata 问题
- 8. Day 4 - Evidence / Citation Fix：修复证据与引用问题
- 9. Day 5 - README + Architecture：强化可迁移表达
- 10. Day 6 - Performance Hygiene：清理指标与补可复现实验记录
- 11. Day 7 - Final QA + Week4 Handoff：收尾、演示、可选二域 demo
- 12. 最终验收清单、面试表达、附录模板

1. 文档定位与 Week 3 RAG 核心判断

Week 3 的主线原本是 RepoPilot v0 + Agent Loop / Tool Calling 核心闭环；RAG Copilot 在 Week 3 只承担维护线。这个事实必须写进 RAG 专属执行手册，否则容易误以为本周还要继续大规模加 RAG 功能。

本周 RAG 的真正目标不是“再做一个高级功能”，而是让 Week 2 已经完成的 Hybrid RAG 工程化成果更可靠、更可解释、更适合进入 Week 4 的项目包装。

维度	Week 2 已经完成或应完成	Week 3 只做什么
系统能力	SQL / structured tools + document retrieval、intent router、evidence、fallback、evaluation、performance tracing。	只修复这些能力暴露出的失败 case，不继续扩散到 GraphRAG、多 Agent、复杂 reranker。
代码重点	intent_router、tools、metadata_rules、retriever、evidence_builder、fallback_handler、performance_tracer。	围绕真实失败样例做局部 patch，不重写 pipeline。
文档重点	evaluation.md、failure_cases.md、performance.md 初版。	补充失败原因、修复记录、指标来源、局限说明。
面试重点	能解释 Hybrid RAG 为什么比普通文档问答更接近企业场景。	能解释自己如何维护和评估 RAG，而不是只会堆功能。

1.1 本周硬性不做事项

- 不继续无节制增加 RAG 新功能。
- 不把 Week 3 做成第二个 Week 2；如果不是来自 failure case 或包装缺口，默认不做。
- 不引入 GraphRAG、复杂 reranker、生产级缓存、全套 observability 平台。
- 不虚构 P95/P99/QPS/cache hit rate；没有脚本或日志来源的数字必须删除。
- 不让 Codex 大范围重构 core pipeline；除非它先提交最小设计说明并通过人工 review。
- 不为了演示效果绕过 fallback / evidence / domain adapter 边界。

2. 实践公约在 Week 3 RAG 维护中的落地方式

Week 3 的 RAG 维护属于“生产区维护”。原因是：这些代码、文档、评估结果和性能记录会进入 README、GitHub、简历和面试追问。维护任务不能只看是否能跑，必须看是否有失败样例、是否有回归测试、是否有文档留痕。

任务类型	区域判断	执行方式	拒绝合并条件
修复 intent 错误	生产区维护	先定位 evaluation case，再局部修 intent_router 或 domain routing。	没有失败 case；直接改 prompt 猜测；影响已有通过 case。
修复 retrieval / metadata 错误	生产区维护	先记录 query、expected evidence、actual evidence，再调整 metadata 或检索参数。	大范围重写 retriever；删除 metadata 规则；没有 regression check。
修复 evidence / citation 错误	生产区维护	统一 structured evidence 和 document evidence 格式，确保回答可追溯。	关键业务回答没有 evidence；generator 编造证据。
修复 fallback 错误	生产区维护	将触发原因写入 fallback_reason，并补 failure_cases.md。	证据不足仍强答；fallback_reason 不可解释。
补 performance.md	生产区文档	只记录可复现脚本、环境、样本、结果和局限。	写入无法复现的指标；夸大本地小样本实验。
可选二域 demo	探索区 / 辅助证明	只做 very small HR/IT support mini docs + 1-2 query。	占用主线时间；改动 core pipeline；试图做完整新业务系统。

2.1 Week 3 RAG 开发硬规则

- 13. RAG 修复必须基于失败样例。没有 failure case，不做功能修复。
- 14. 业务规则仍然不能写死在 core pipeline。电商规则留在 domains/ecommerce/。
- 15. 所有关键回答必须保留 evidence；没有 evidence 的业务回答必须 fallback。
- 16. 任何性能数字必须有命令、环境、样本数量或日志来源。
- 17. Codex 生成代码默认是候选稿；Chenning 必须 review 后才能合并。
- 18. 每天只允许 1-2 个小 patch，优先修高优先级失败，不做大范围重构。
- 19. 每天必须更新 failure_cases.md 或相关文档，否则维护任务不算完成。
- 20. 本周 RAG 至少 2 条维护相关 commit；每条 commit message 必须说明修复了哪个 failure case 或文档缺口。

3. Week 3 RAG 总交付物与目录边界

如果 Week 1/2 的目录和下面不完全一致，不要强行重建。Codex 需要按“职责”寻找对应文件，而不是按文件名机械操作。

```
enterprise-rag-copilot/  
  pipeline/  
    intent_router.py  
    retriever.py  
    evidence_builder.py  
    fallback_handler.py  
    prompt_builder.py
```

```

answer_generator.py
performance_tracer.py
domains/ecommerce/
  schema.py
  repository.py
  tools.py
  metadata_rules.py
  adapter.py
evaluation/
  ecommerce_cases.json
  run_eval.py
  metrics.py
docs/
  architecture.md
  api.md
  evaluation.md
  failure_cases.md
  performance.md
  demo_queries.md          # optional, useful for Week 4
  week3_rag_maintenance_plan.md
  week4_packaging_todo.md
scripts/
  quick_latency_check.py   # optional if load_test.py exists
  load_test.py
tests/
  test_intent_router.py
  test_retrieval_metadata.py
  test_evidence_builder.py
  test_fallback_handler.py
README.md

```

最终交付物	最低要求	验收方式
docs/failure_cases.md	至少 5 个 Week2 失败 case; 至少 3 个已修复; 每个 case 有 root cause、fix、verification。	人工阅读 + run_eval 对应 case 通过。
局部代码 patch	至少修复 3 个高优先级失败 case。	对应测试或 eval case 通过, 不破坏已有核心 demo。
README.md	有 quick start、API example、demo query、Week2/3 能力说明。	新读者可以按 README 启动并跑 1 个 query。
docs/architecture.md	更清楚表达 core pipeline + domain adapter; 说明可迁移性。	能用 2 分钟讲清不是写死的电商客服。
docs/evaluation.md	说明 case 类型、指标含义、如何运行、结果局限。	命令可执行或至少语义明确。
docs/performance.md	写清测试环境、命令、样本、结果、局限; 删除虚假指标。	每个数字有来源。
Week4 handoff	列出 Week4 包装还缺什么。	docs/week4_packaging_todo.md 完成。

4. Codex 工作协议

每一天给 Codex 的任务都必须套用以下协议。不要只告诉 Codex “帮我修一下 RAG”，否则它很容易大范围乱改。

Context:

你正在维护 enterprise-rag-copilot。Week 3 RAG 不是主开发线，只允许基于 Week 2 evaluation/failure cases 做小修复、补文档和清理指标。

Goal:

本次只完成一个可 review 的 RAG 维护任务：<填写今天唯一主目标>。

Constraints:

- 不重写 core pipeline。
- 不引入新框架或重依赖。
- 不虚构性能指标。
- 不把电商业务规则写进 pipeline/ 通用层。
- 生成的代码是候选稿，必须附带修改文件、原因、验收命令和风险。

Interface & Data:

失败样例来自 docs/failure_cases.md 或 evaluation/ecommerce_cases.json。

请保持 ChatResponse / Evidence / fallback_reason 的既有契约稳定。

Process:

先读相关文件 -> 解释 root cause -> 给最小修改方案 -> 修改代码/文档 -> 给测试命令 -> 给 Review Checklist。

Review:

如果没有失败样例、没有验收命令、没有文档留痕，拒绝合并。

4.1 每次 Codex 输出必须包含

- Changed files：列出修改了哪些文件。
- Root cause：说明修复的是 intent / retrieval / evidence / fallback / doc / performance 哪类问题。
- Why minimal：说明为什么不是大范围重构。
- Verification：给出 pytest、run_eval 或手动 curl 命令。
- Risk：说明可能影响哪些已有 case。
- Follow-up：如果只是暂时修复，要写后续待办。

5. Day 1 - Failure Triage：整理 Week 2 失败样例

今日目标：把 Week 2 的 evaluation / failure cases 从“零散记录”整理成 Week 3 的修复队列。今天不急着想代码，先决定本周到底修什么。

A. 今日允许创建或修改的文件

- docs/failure_cases.md
- evaluation/ecommerce_cases.json

- evaluation/run_eval.py
- docs/week3_rag_maintenance_plan.md
- reports/week3_eval_baseline.json (可选)

B. 给 Codex 的任务卡

读取 docs/failure_cases.md 和 evaluation/ecommerce_cases.json，整理 3-5 个最高优先级 failure cases。为每个 case 标注 category、priority、expected、actual、root_cause_hypothesis、planned_fix 和 verification_command。不要修改业务代码。

C. 人工执行步骤

- 21. 运行 Week2 eval baseline: python evaluation/run_eval.py --suite evaluation/ecommerce_cases.json --output reports/week3_eval_baseline.json。若脚本尚未支持 output 参数，先手动保存结果摘要。
- 22. 打开 docs/failure_cases.md，补齐至少 5 个 failure case。
- 23. 按影响演示/面试的程度排序：P0 影响核心 demo；P1 影响常见 query；P2 仅影响边缘 case。
- 24. 把本周只修 3 个高优先级 case 写进 docs/week3_rag_maintenance_plan.md。
- 25. 写清本周不做事项：不做新 RAG 框架，不做复杂 rerank，不做大重构。

D. 今日验收标准

- ☐ docs/failure_cases.md 至少有 5 个 case。
- ☐ docs/week3_rag_maintenance_plan.md 明确本周修复范围。
- ☐ 每个待修 case 有 verification command。
- ☐ 没有任何业务代码大改。

E. Review Gate

检查项	通过标准
Failure first	本次改动必须能对应到 docs/failure_cases.md 中的某个 case。
Minimal patch	只修改必要文件，不做跨模块大重构。
Contract stable	不破坏 ChatRequest / ChatResponse / Evidence / fallback_reason 的外部契约。
Verification	有 pytest、run_eval、curl 或手动验收命令。
Documentation	failure_cases.md、evaluation.md、performance.md、README 或 architecture 中至少一个有同步更新。

F. 建议 commit message 与面试表达

建议 commit: docs(rag): triage week2 failure cases for week3 maintenance

今日面试表达: Week 3 我没有盲目堆 RAG 功能，而是先复盘 Week 2 evaluation，按失败影响排序，选择最影响演示和面试的 case 做局部修复。

6. Day 2 - Intent / Fallback Fix：修复路由与兜底高优先级问题

今日目标：修复第一个高优先级失败：常见是 unknown intent、missing_order_id、hybrid query 路由错误或 fallback_reason 不清楚。

A. 今日允许创建或修改的文件

- pipeline/intent_router.py
- pipeline/fallback_handler.py
- schemas/chat.py 或 schemas/evidence.py
- evaluation/ecommerce_cases.json
- tests/test_intent_router.py
- tests/test_fallback_handler.py
- docs/failure_cases.md

B. 给 Codex 的任务卡

只修复一个 intent/fallback 类 failure case。请先定位失败 case，再提出最小 patch。保持 ChatResponse 字段兼容，不改变 /chat 外部契约。补一个 regression test 或 eval case。

C. 人工执行步骤

- 26. 从 Day1 队列选择一个 intent/fallback 类 P0/P1 case。
- 27. 确认输入 query、expected_intent、actual_intent、expected_fallback。
- 28. 局部调整 intent_router 的规则或 fallback_handler 的触发条件。
- 29. 补充或修正 evaluation case，确保 should_fallback / expected_route 清楚。
- 30. 添加最小测试：pytest tests/test_intent_router.py tests/test_fallback_handler.py。
- 31. 更新 docs/failure_cases.md：status 从 open 改为 fixed 或 partially_fixed。

D. 今日验收标准

- [] 对应 failure case 重新通过。
- [] fallback_reason 可读，不只是 unknown_error。
- [] 没有把电商规则写进 controller 或 answer_generator。
- [] 新增/修正测试能复现旧错误。

E. Review Gate

检查项	通过标准
Failure first	本次改动必须能对应到 docs/failure_cases.md 中的某个 case。
Minimal patch	只修改必要文件，不做跨模块大重构。
Contract stable	不破坏 ChatRequest / ChatResponse / Evidence / fallback_reason 的外部契约。
Verification	有 pytest、run_eval、curl 或手动验收命令。
Documentation	failure_cases.md、evaluation.md、performance.md、README 或 architecture 中至少一个有同步更新。

F. 建议 commit message 与面试表达

建议 commit: fix(rag): repair high-priority intent fallback case

今日面试表达: 我把 fallback 当成可靠性能能力，而不是错误兜底文本；缺订单号、未知意图、证据不足都要有明确 fallback_reason 和建议动作。

7. Day 3 - Retrieval / Metadata Fix: 修复检索与 metadata 问题

今日目标：修复第二个高优先级失败：文档类型过滤错误、policy version 不生效、query 找不到正确 policy chunk、低分检索未触发 fallback。

A. 今日允许创建或修改的文件

- domains/ecommerce/metadata_rules.py
- pipeline/retriever.py
- pipeline/reranker.py (如已有 mock 接口，只做兼容检查)
- pipeline/document_loader.py 或 pipeline/chunker.py
- data/ecommerce/docs/*.md
- tests/test_retrieval_metadata.py
- docs/failure_cases.md

B. 给 Codex 的任务卡

只修复一个 retrieval/metadata 类 failure case。不要替换整个 vector store。优先检查 chunk metadata 是否丢失、document_type 是否不匹配、intent 到 filter 的映射是否错误。

C. 人工执行步骤

- 32. 从 Day1 队列选择一个 retrieval/metadata 类 case。
- 33. 记录 expected evidence keywords 和 actual retrieved chunks。
- 34. 检查 document_type / product_category / policy_version / applicable_scenario 是否在 chunk 中保留。
- 35. 局部修 metadata_rules 或 retriever filter mapping。
- 36. 如果 query rewrite 是原因，只做最小规则，不写复杂 prompt。
- 37. 补测试：输入 query，断言 top-k 中包含预期 source 或关键词。
- 38. 更新 docs/failure_cases.md。

D. 今日验收标准

- [] 目标 query 能检索到正确 policy 文档或 chunk。
- [] metadata filter 不影响其他文档问题。
- [] 没有新依赖，没有重写索引层。
- [] failure case 有 before/after 记录。

E. Review Gate

检查项	通过标准
Failure first	本次改动必须能对应到 docs/failure_cases.md 中的某个 case。
Minimal patch	只修改必要文件，不做跨模块大重构。
Contract stable	不破坏 ChatRequest / ChatResponse / Evidence / fallback_reason 的外部契约。
Verification	有 pytest、run_eval、curl 或手动验收命令。
Documentation	failure_cases.md、evaluation.md、performance.md、README 或 architecture 中至少一个有同步更新。

F. 建议 commit message 与面试表达

建议 commit: fix(rag): improve metadata filtering for policy retrieval

今日面试表达: metadata filtering 的价值是让检索先缩小到正确文档类型和适用场景，再做相似度匹配，避免纯向量检索召回看似相关但业务上不适用的内容。

8. Day 4 - Evidence / Citation Fix: 修复证据融合与引用问题

今日目标: 修复第三个高优先级失败: answer 有内容但 evidence 缺失、structured evidence 和 document evidence 混乱、引用来源不清楚。

A. 今日允许创建或修改的文件

- schemas/evidence.py
- pipeline/evidence_builder.py
- pipeline/prompt_builder.py
- pipeline/answer_generator.py
- tests/test_evidence_builder.py
- docs/evaluation.md
- docs/failure_cases.md

B. 给 Codex 的任务卡

只修复一个 evidence/citation 类 failure case。统一 structured_evidence 与 document_evidence 的最小格式，不允许 answer_generator 编造证据；没有 evidence 时必须触发 fallback 或降级表达。

C. 人工执行步骤

39. 从 Day1 队列选择一个 evidence/citation 类 case。
40. 检查 /chat response 中 evidence list 是否为空、source 是否缺失、score/type 是否不一致。
41. 统一 Evidence 字段: type、source、content、score、metadata、tool_name (可选)。
42. 让 evidence_builder 接收 structured tool result 和 retrieved chunks，输出统一列表。
43. 在 prompt_builder 中明确“回答必须基于 evidence”。
44. 补测试: structured-only、document-only、hybrid 三类 evidence。
45. 更新 docs/evaluation.md 中 citation correctness 的判断方式。

D. 今日验收标准

- [] 关键业务回答一定返回 evidence list。
- [] hybrid 问题能同时保留 tool evidence 和 document evidence。
- [] 没有 evidence 时不会强答。
- [] docs/evaluation.md 说明 citation correctness 如何人工/脚本检查。

E. Review Gate

检查项	通过标准
Failure first	本次改动必须能对应到 docs/failure_cases.md 中的某个 case。
Minimal patch	只修改必要文件，不做跨模块大重构。
Contract stable	不破坏 ChatRequest / ChatResponse / Evidence / fallback_reason 的外部契约。
Verification	有 pytest、run_eval、curl 或手动验收命令。
Documentation	failure_cases.md、evaluation.md、performance.md、README 或 architecture 中至少一个有同步更新。

F. 建议 commit message 与面试表达

建议 commit: fix(rag): normalize evidence builder and citation behavior

今日面试表达: 我把 evidence 设计成可统一表达数据库结果和文档片段的结构，这样后续可以评估 citation correctness，而不是只看回答文字像不像。

9. Day 5 - README + Architecture：强化可迁移表达

今日目标：把 RAG 项目从“能跑”整理成“别人能看懂、能启动、能理解架构取舍”的项目。今天以文档为主，少改代码。

A. 今日允许创建或修改的文件

- README.md
- docs/architecture.md
- docs/api.md
- docs/demo_queries.md（可选）
- docs/week4_packaging_todo.md

B. 给 Codex 的任务卡

根据现有代码和 Week2/Week3 能力更新 README 与 architecture 文档。不要虚构未实现功能。必须区分已完成、可选、后续计划。重点表达 core pipeline + domain adapter，而不是单一电商客服 demo。

C. 人工执行步骤

- 46. README 增加或检查：Project Motivation、Architecture、Quick Start、API Example、Demo Queries、Evaluation、Performance、Known Limitations。
- 47. docs/architecture.md 增加“Core Pipeline vs Ecommerce Domain Adapter”小节。
- 48. docs/api.md 固定 /health、/chat、/retrieve（如存在）的请求/响应例子。
- 49. docs/demo_queries.md 准备 5-8 个稳定 query：order、refund、policy、hybrid、fallback。
- 50. docs/week4_packaging_todo.md 列出 Week4 还要补的 traces、benchmarks、interview_notes、resume_bullets。

D. 今日验收标准

- [] README 能让新读者启动项目并跑至少 1 个 demo query。
- [] architecture 文档能解释为什么可迁移。
- [] 没有写“生产级”“高并发”等未支撑表达。
- [] API 示例与当前 schema 一致。

E. Review Gate

检查项	通过标准
Failure first	本次改动必须能对应到 docs/failure_cases.md 中的某个 case。
Minimal patch	只修改必要文件，不做跨模块大重构。
Contract stable	不破坏 ChatRequest / ChatResponse / Evidence / fallback_reason 的外部契约。
Verification	有 pytest、run_eval、curl 或手动验收命令。
Documentation	failure_cases.md、evaluation.md、performance.md、README 或 architecture 中至少一个有同步更新。

F. 建议 commit message 与面试表达

建议 commit：docs(rag): strengthen README and architecture for transferable RAG

RAG Copilot Week 3 专属执行手册 | 生产区维护：failure first, minimal patch, reproducible metrics

今日面试表达：我没有把项目包装成单一电商客服机器人，而是把电商作为 domain adapter；core pipeline 保持可迁移，后续可以替换到 HR、IT support 等企业知识场景。

10. Day 6 - Performance Hygiene：清理指标与补可复现实验记录

今日目标：检查 performance.md，删除不能复现或容易夸大的数字；补最小本地检查脚本或命令。

A. 今日允许创建或修改的文件

- docs/performance.md
- pipeline/performance_tracer.py
- scripts/load_test.py 或 scripts/quick_latency_check.py
- benchmarks/week3_local_latency_check.md（可选）
- traces/sample_*.json（如已有 trace 输出）

B. 给 Codex 的任务卡

只做 performance 文档和最小脚本补强。不得生成虚假 P95/P99/QPS。若没有真实运行结果，就写“待测”并保留命令模板。检查 performance_tracer 字段是否能支撑后续实验。

C. 人工执行步骤

- 51. 打开 docs/performance.md，逐条检查所有数字是否有来源。
- 52. 保留或补充测试环境：机器、Python 版本、LLM 模式、数据规模、接口、样本数量。
- 53. 确认 performance_tracer 至少记录：intent_time_ms、structured_tool_time_ms、retrieval_time_ms、llm_time_ms、total_time_ms。
- 54. 如果已有 load_test.py，补 usage；否则创建 quick_latency_check.py 只请求 /health 和 /chat mock。
- 55. 写明本地实验局限：小样本、mock LLM、非生产部署、无真实并发环境。

D. 今日验收标准

- [] performance.md 中每个数字都有命令或日志来源。
- [] 无法验证的指标被删除或标记为待测。
- [] 至少保留一个可运行的本地测试命令模板。
- [] 文档明确说明不能将本地小样本夸大为生产能力。

E. Review Gate

检查项	通过标准
Failure first	本次改动必须能对应到 docs/failure_cases.md 中的某个 case。
Minimal patch	只修改必要文件，不做跨模块大重构。
Contract stable	不破坏 ChatRequest / ChatResponse / Evidence / fallback_reason 的外部契约。
Verification	有 pytest、run_eval、curl 或手动验收命令。
Documentation	failure_cases.md、evaluation.md、performance.md、README 或 architecture 中至少一个有同步更新。

F. 建议 commit message 与面试表达

建议 commit: docs(rag): audit performance report and reproducible metrics

今日面试表达: 我在 performance 文档里明确区分已测结果和待测指标，避免为了简历好看虚构 P95/P99 或 QPS；所有数字都必须能回到脚本、日志和测试环境。

11. Day 7 - Final QA + Week4 Handoff: 收尾、演示、可选二域 demo

今日目标：完成 Week 3 RAG 维护最终验收，确保 Week 4 可以直接进入项目包装。可选做 very small HR/IT support demo，但只有所有必做项完成后才允许。

A. 今日允许创建或修改的文件

- docs/week3_rag_summary.md
- docs/week4_packaging_todo.md
- docs/failure_cases.md
- README.md
- data/hr_it/docs/*.md (可选)
- domains/hr_it/adapter.py (可选，极小)

B. 给 Codex 的任务卡

先完成 Week3 RAG final QA 和 handoff 文档。只有全部必做验收通过后，才可以创建 very small HR/IT support mini docs + 1-2 demo query，用于证明 core pipeline 可迁移；不得改 core pipeline。

C. 人工执行步骤

- 56. 运行最小验收：pytest tests/test_intent_router.py tests/test_retrieval_metadata.py tests/test_evidence_builder.py tests/test_fallback_handler.py。
- 57. 运行 evaluation：python evaluation/run_eval.py --suite evaluation/ecommerce_cases.json。
- 58. 手动跑 5 个 demo query：order、refund、policy、hybrid、fallback。
- 59. 更新 docs/week3_rag_summary.md：本周修复了什么、未修什么、风险是什么。
- 60. 更新 docs/week4_packaging_todo.md：README、api、sample traces、demo script、interview notes、resume bullets 还缺什么。
- 61. 可选：新增 very small HR/IT support mini docs，只写 1-2 个问答样例，证明 domain adapter 思路，不做完整新系统。

D. 今日验收标准

- [] 至少 3 个高优先级 failure cases 已修复或明确标为 postponed。
- [] README、architecture、evaluation、performance、failure_cases 均无明显虚假或过期内容。
- [] 可以用 2 分钟演示 SQL + 文档 RAG 混合问答。
- [] Week4 packaging todo 可直接执行。
- [] 可选二域 demo 没有污染 core pipeline。

E. Review Gate

检查项	通过标准
Failure first	本次改动必须能对应到 docs/failure_cases.md 中的某个 case。
Minimal patch	只修改必要文件，不做跨模块大重构。
Contract stable	不破坏 ChatRequest / ChatResponse / Evidence / fallback_reason 的外部契约。

Verification	有 pytest、run_eval、curl 或手动验收命令。
Documentation	failure_cases.md、evaluation.md、performance.md、README 或 architecture 中至少一个有同步更新。

F. 建议 commit message 与面试表达

建议 commit: chore(rag): finalize week3 maintenance and week4 handoff

今日面试表达: Week 3 的 RAG 工作体现的是维护能力：我基于 evaluation 修复失败样例，清理性能指标，强化 README 和架构表达，为 Week 4 的项目包装做准备。

12. Week 3 RAG 最终验收清单

类别	验收项
Failure Cases	[] Week 2 failure cases 已整理；[] 至少 5 个 case 有 root cause；[] 至少 3 个高优先级 case 已修复或明确 postponed。
Regression	[] 修复过的 case 有测试或 eval 覆盖；[] 旧的核心 demo query 没有被破坏。
README	[] quick start 可用；[] API example 可用；[] demo query 可用；[] 写清已实现与未实现。
Architecture	[] core pipeline + domain adapter 表达更清楚；[] 说明为什么不是写死电商客服。
Evaluation	[] evaluation.md 解释指标；[] ecommerce_cases.json 至少 30 条仍可用；[] run_eval 命令清楚。
Performance	[] performance.md 没有虚构指标；[] 所有数字有环境和命令来源；[] 局限写清楚。
Evidence/Fallback	[] 关键回答带 evidence；[] 证据不足触发 fallback；[] fallback_reason 可解释。
Week4 Handoff	[] week4_packaging_todo.md 完成；[] 可以直接进入 demo_script、interview_notes、resume bullets。
Commit	[] 至少 2 条 RAG 维护相关 commit；[] 每条 commit 能对应到修复、文档或指标清理。

13. RAG 维护追问 Q&A 初版

Q1: Week 3 为什么不继续给 RAG 加新功能？

答：因为 Week 3 的 RAG 任务是维护和补强。Week 2 已经完成 Hybrid RAG 的核心工程化能力，本周应基于 evaluation 修复失败样例、补文档和清指标，为 Week 4 包装做准备。

Q2: 你如何决定先修哪些 RAG 失败 case？

答：我按演示影响和面试追问风险排序。影响核心 demo 的 intent、evidence、fallback 错误优先级最高，边缘 query 或可选优化放到 postponed。

Q3: 为什么 failure_cases.md 有价值？

答：它记录系统失败原因、修复方式和验证命令，说明项目不是只展示成功 demo，也能分析局限并持续迭代。

Q4：你如何避免 RAG 编造答案？

答：关键回答必须绑定 structured evidence 或 document evidence；缺证据、低置信度、缺订单号或高风险问题会 fallback，而不是强答。

Q5：Week 3 如何体现工程化维护能力？

答：我没有大改系统，而是基于失败样例做最小 patch，同时补 regression test、文档和性能指标来源。

Q6：performance.md 为什么不能随便写数字？

答：性能指标会进入简历和面试追问。所有 P95/P99/QPS/cache hit rate 必须来自脚本、日志和明确测试环境，否则就是虚假包装。

Q7：什么是 core pipeline + domain adapter？

答：core pipeline 负责通用流程，如 routing、retrieval、evidence、fallback；domain adapter 承载电商订单、退款、物流和保修规则。这样后续可以迁移到 HR/IT support 等场景。

Q8：可选二域 demo 的意义是什么？

答：它只用于证明核心管线不被电商写死，但必须 very small，不能占用主线时间或污染 core pipeline。

Q9：Week 3 结束后 RAG 应该达到什么状态？

答：能稳定演示 SQL + 文档 RAG 混合问答，失败 case 有记录，关键文档可读，指标不虚构，可以进入 Week 4 的项目包装。

Q10：如果 Codex 生成了大范围重构怎么办？

答：拒绝合并。Week 3 RAG 只接受基于 failure case 的最小 patch，必须有验收命令和文档留痕。

14. 附录 A：failure_cases.md 推荐格式

```
## FC-003: return policy query retrieves warranty policy
- Category: retrieval / metadata
- Priority: P1
- Query: 这件商品超过 7 天还能退吗？
- Expected intent: return_policy
- Expected evidence: return_policy.md, 7-day return rule
- Actual behavior: retrieved warranty_policy.md chunk
- Root cause hypothesis: document_type metadata missing after chunking
- Fix: preserve document_type in split_documents(); add intent->metadata filter
- Verification:
```

- pytest tests/test_retrieval_metadata.py -k return_policy
- python evaluation/run_eval.py --case FC-003
- Status: fixed / partially_fixed / postponed
- Notes: keep as regression case for Week4 demo.

15. 附录 B: docs/performance.md 最小可信格式

```
# Performance Notes - Week 3 RAG Maintenance

## Environment
- Machine: <local laptop / cloud instance>
- Python: <version>
- LLM mode: mock / API
- Dataset size: <docs/chunks/cases>
- Server command: uvicorn app.main:app --reload

## Commands
- python scripts/quick_latency_check.py --endpoint http://localhost:8000/chat --n 20
- python evaluation/run_eval.py --suite evaluation/ecommerce_cases.json

## Results
| Metric | Value | Source |
|---|---:|---|
| avg_total_latency_ms | <measured> | quick_latency_check.py |
| p95_total_latency_ms | <measured or TODO> | quick_latency_check.py |
| error_rate | <measured> | quick_latency_check.py |

## Limitations
- Local machine only, not production deployment.
- Small sample size.
- Mock LLM if applicable.
- Do not present these as production QPS/P95 numbers.
```

16. 附录 C: Week 4 Handoff 模板

```
# Week 4 RAG Packaging TODO

## Must finish
- [ ] README final polish
- [ ] docs/api.md examples verified
- [ ] docs/demo_script.md: 3-5 min demo
- [ ] docs/interview_notes.md: at least 20 Q&A
- [ ] traces/: 3 sample traces (order, policy, fallback)
- [ ] benchmarks/: local latency report
- [ ] resume_bullets.md: 3-5 bullets, no fake metrics

## Known limitations to be honest about
```

- [] no production-grade vector DB benchmark yet
- [] reranker is mock/interface only, if applicable
- [] cache demo is local/simple only
- [] evaluation cases are small but structured

Best demo queries

1. <order status query>
2. <refund status query>
3. <return policy query>
4. <hybrid order + policy query>
5. <fallback query>

17. Week 3 RAG 最重要的一句话

Week 3 的 RAG 不是为了证明你还能继续堆功能，而是为了证明：你能基于 evaluation 和 failure cases 维护一个 RAG 工程项目，修复高优先级失败，保留 evidence 与 fallback 的可靠性边界，清理不可复现指标，并把项目准备好进入 Week 4 的包装、面试和简历表达。